

MediClaim Orchestrator

Technology Stack

What we use, why we use it, and how to run it

Companion to the project report

Repository: github.com/Multi-Agent-MediClaim-Orchestrator/Sohail-Ahmed

Prepared by: the project team

October 2026 · Version 1.0

1. The stack at a glance

The system has five layers. **Agents** (CrewAI) do the reading and writing that needs language understanding. **Workflows** (n8n) run the business process: triggers, waits, retries, SLAs. **Services** (FastAPI) own the data and every rule. **AI runtime** (Ollama, Qdrant) runs the models locally. **Infrastructure** (Docker Compose) gives every part a database, a cache, a file store, a virus scanner and an identity provider. Everything is open source and runs on one laptop, so no patient data leaves the machine and no paid API key is needed.

Table 1: Technology summary

Layer	Technology	Version	Problem it solves
Agents	CrewAI (Agents, Tasks, Crews, Flows)	1.15.x	Turns each AI job into a named agent with a role, a task and a guardrail; Flows give a visible, testable order of steps
LLM runtime	Ollama + gemma4, nomic-embed-text	latest	Runs the language and embedding models locally: no cost per call, no data leaves the machine
Workflows	n8n	2.41.6	Business process automation (webhooks, schedules, retries, SLA timers) that a non-programmer can read; 13 hospital + 19 insurer workflows
Backend	Python 3.12, FastAPI, Pydantic, SQLAlchemy, Alembic	3.12 / 0.11x / 2.x	Typed REST APIs with automatic validation and OpenAPI docs; versioned database migrations
Frontend	Next.js, React, TypeScript	14	Hospital desk and insurer reviewer screens
Databases	PostgreSQL ×2	16	One database per organisation, with row-level roles and a tamper-evident audit chain
Vector store	Qdrant + own hybrid search and reranker	1.12.4	Finds policy wording and rules to cite (RAG); separate collections per organisation
Cache / events	Redis	7	Idempotency keys, rate limits, the event stream between services
Object storage	MinIO (built from source)	2025-04-22	S3-compatible store for scanned documents, encrypted at rest
Security	ClamAV, Keycloak, Presidio, HMAC, JWT	stable / 25	Virus scan on upload, user login and roles, PII masking before any model call, signed API calls between organisations
Documents	Poppler, Tesseract, OpenCV	system	PDF to image, OCR of scanned bills, stamp and quality checks
Tooling	uv, Docker Compose, Make, pytest, ruff, mypy, Playwright, Mermaid	-	One-command build and run, tests, lint, browser tests, diagrams

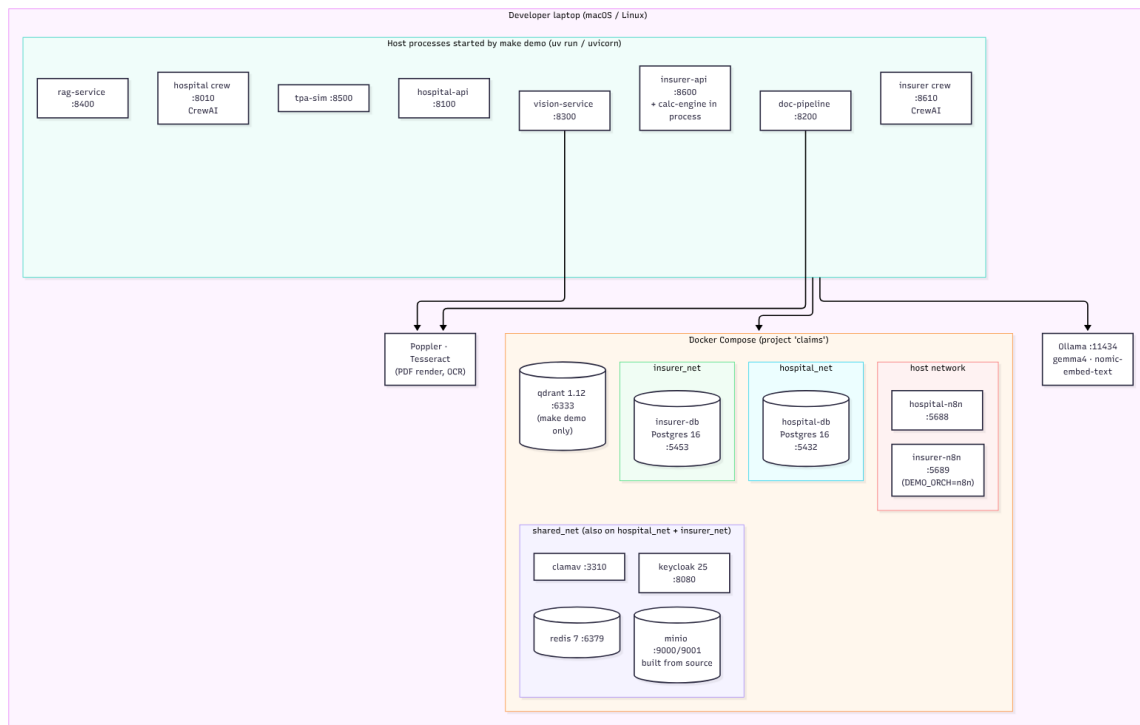


Figure 1: Deployment view: containers (Docker Compose) and host processes

2. Each technology: what, why, and its parameters

2.1 CrewAI: the agent framework

What. A Python framework for multi-agent systems. An **Agent** has a role, goal and backstory; a **Task** is a piece of work with an expected output; a **Crew** runs tasks with agents; a **Flow** is an event-driven state machine (@start, @listen, @router) that calls crews and plain Python in a fixed order.

Why. The project guidelines require a multi-agent framework. The original code called the model from hand-written Python per agent; CrewAI makes each agent explicit (defined in `config/agents.yaml`), adds **guardrails** that reject and retry a bad answer, and lets us draw the flows (`make crew-plot`).

Where. Hospital: ClaimFlow (build or repair the claim, estimate the admissible amount, post the draft) and QueryFlow (triage, draft a reply). Insurer: VerificationFlow (identity, authenticity, coverage, calculation mapping, supervisor) and QueryFlow (draft a query, triage the hospital's reply).

Parameter	Meaning
<code>CREW_LLM = ollama rules</code>	Hospital crew: real model, or rule-based stand-in (no model)
<code>INS_CREW_LLM = ollama offline</code>	Insurer crew: real model, or offline stand-in
<code>HOSP_LLM_BASE_URL,</code> <code>INS_CREW_MODEL</code>	Model endpoint (Ollama, OpenAI-compatible) and model name (default gemma4:latest)
<code>guardrail,</code> <code>guardrail_max_retries</code>	Function that checks the agent's output; the task is retried with feedback if it fails
<code>max_retry_limit=0, memory off</code>	Errors are raised to our code, not hidden; no hidden state between claims
<code>CREWAI_TELEMETRY_OPT_OUT=true</code>	No telemetry leaves the machine (set in the Makefile and .env)

2.2 Ollama and the local models

What. A local model server with an OpenAI-compatible API on port 11434. **gemma4** writes and reasons; **nomic-embed-text** turns text into vectors for search. **Why.** Health data is sensitive personal data under India's DPDP Act; running locally means nothing is sent to a cloud provider, it costs nothing per call, and it works offline. **Parameters:** `OLLAMA_HOST`, `RAG_CHAT_MODEL`, `RAG_EMBED_MODEL`, `reasoning_effort=none` (faster answers), per-call timeouts.

2.3 n8n: workflow automation

What. A visual workflow engine (webhooks, schedules, HTTP calls, retries, error branches). **Why.** Claims are long-running processes with waits (queries, approvals, SLAs). n8n runs those processes and keeps them visible; the decisions stay in tested code that n8n calls. Workflows are generated from code (`build_flows.py`), linted and imported automatically, so they are versioned in Git. **Parameters:** ports 5688 (hospital) and 5689 (insurer), `N8N_ENCRYPTION_KEY`, webhook secrets, the error-handler workflow, and the `u1-u3` sub-workflows for service tokens, idempotency and polling.

2.4 FastAPI, Pydantic, SQLAlchemy, Alembic

What. FastAPI is an async Python web framework; Pydantic validates every request and response against a schema; SQLAlchemy maps tables to Python; Alembic versions the schema (the hospital database is at migration `0023`). **Why.** Typed contracts catch bad data at the edge, the OpenAPI file is generated and checked in CI, and migrations make the database reproducible. **Services and ports:** hospital-api 8100, doc-pipeline 8200, vision-service 8300, rag-service 8400, tpa-sim 8500, insurer-api 8600, insurer-crew 8610, calc-engine 8620, hospital crew 8010.

2.5 Calculation engine and decision gate

What. A pure-Python engine that computes the payable amount with `Decimal` (never floating point): room-rent capping with proportionate deduction, sub-limits, co-pay, deductibles, non-payables, sum-insured balance. **Why.** Money must be exact and explainable, so no model ever decides an amount. **Parameters:** product rules (for example HEALTH-PLUS-GOLD: room 1% of sum insured, ICU 2%, co-pay 0%), and the gate thresholds `T_auto = ₹50,000` (below it a clean claim can auto-approve) and `T_four = ₹5,00,000` (above it two approvers are needed).

2.6 Qdrant and the RAG service

What. Qdrant is a vector database; our rag-service adds hybrid dense+sparse search and a reranker. **Why.** Agents must quote the actual policy wording, not invent it. Collections are split by organisation (`ins_*`, `hosp_*`) and each caller gets a JWT that only opens its own collections. **Parameters:** `QDRANT_URL` (6333), `QDRANT_API_KEY`, `RAG_JWT_SECRET`, `RAG_PORT` 8400, `RAG_SEED_ON_START`, top-k and rerank settings.

2.7 PostgreSQL, Redis, MinIO

PostgreSQL 16 (hospital 5432, insurer 5453): one database per organisation, separate owner/app/read-only roles, hash-chained audit table. **Redis 7** (6379): idempotency keys, rate limits, the `sse:insurer:events` stream; each service has its own Redis user and password. **MinIO** (9000, console 9001): S3-compatible document storage with server-side encryption (`MINIO_KMS_SECRET_KEY`) and one access key per service. We build MinIO from source because the official image was removed from Docker Hub.

2.8 Security components

- **ClamAV** (clamav-debian, multi-arch): every uploaded file is scanned before anything reads it.
- **Keycloak 25** (8080, management 9090): login, roles (desk, approver, admin) and service clients for both organisations.

- **Presidio + our PII guard:** names, phone numbers, Aadhaar-like numbers are masked before any text reaches a model.
- **HMAC-signed calls** (`HOSP_TO_INS_HMAC_SECRET`, `INS_TO_HOSP_HMAC_SECRET`) between hospital and insurer; replay protection with timestamps and idempotency keys.
- **Secrets** are generated by `make init-secrets` into `.env` (never committed).

2.9 Document intelligence

Poppler (`pdftoppm`) turns PDFs into page images; **Tesseract** does OCR; **OpenCV** and a small trained model check stamps and image quality (vision-service). The **doc-pipeline** classifies each document, extracts fields with rules first and the model second, and records a confidence for every field. These two system tools must be installed on the host (`brew install poppler tesseract` on macOS).

2.10 Frontend and tooling

Next.js 14 / React / TypeScript: hospital desk (3100) and insurer reviewer UI (3600), including the new estimate card. **uv** manages one Python workspace for all packages (`uv sync --all-packages`). **Docker Compose** with profiles (`infra`, `hospital`, `insurer`, `rag`) starts only what is needed. **pytest**, **ruff**, **mypy**, **Playwright** test, lint, type-check and drive the browser. **Mermaid** renders the diagrams.

3. Commands: what each one does

All commands run from the repository root. The demo commands are idempotent: running them twice is safe, and every step writes a log to `.e2e-logs/demo/`.

Table 2: One-command targets

Command	What it does	When to use it
<code>make demo</code>	Checks prerequisites, starts all containers, migrates and seeds both databases, builds the knowledge base, starts both crews and all services, imports the n8n workflows, runs end-to-end claims on the real local model and prints the evaluation	The full live demonstration
<code>make demo-offline</code>	The same pipeline with rule-based / stand-in models (no Ollama needed); takes about a minute	Backup for the viva, or a machine without models
<code>make demo-check</code>	Prerequisites only: Docker, uv, make, Poppler/Tesseract, RAM, free ports, Ollama models	Before the first run
<code>make demo-down</code>	Stops the demo's background services and all containers; data volumes are kept	After a demo
<code>make crew-demo</code>	Runs the hospital CrewAI flows on built-in synthetic data, no API or database	To show the agents alone
<code>make ins-crew-demo</code>	Runs the insurer VerificationFlow and QueryFlow on built-in synthetic claims	To show the insurer agents alone
<code>make crew-plot</code>	Writes interactive HTML diagrams of all four CrewAI flows to <code>docs/diagrams</code>	To explain the flow steps
<code>make diagrams</code>	Renders the Mermaid sources to PNG and SVG in <code>docs/images</code>	After editing a diagram

Table 3: Building-block targets (what `make demo` calls internally)

Command	What it does
<code>make init</code> / <code>make init-secrets</code>	Install all Python packages (uv); create <code>.env</code> from <code>.env.example</code> and generate every secret
<code>make up-infra</code>	Render config, start the infra profile (Postgres, Redis, MinIO, ClamAV,

Command	What it does
	Keycloak) and wait until healthy, then create MinIO buckets and keys
<code>make up-insurer</code>	Start the insurer Postgres
<code>make up-n8n / make up-insurer-n8n</code>	Start the hospital / insurer n8n containers; workflows are imported on start
<code>make up-rag / make seed-kb / make run-rag</code>	Start Qdrant; embed and load the synthetic knowledge base; run the rag-service on 8400
<code>make migrate seed / make migrate-insurer seed-insurer</code>	Apply Alembic migrations and load demo data for each side
<code>make run-api, run-ui, run-crew</code>	Hospital API (8100), UI (3100), crew (8010)
<code>make run-insurer-api, run-insurer-crew, run-calc, run-tpa-sim</code>	Insurer API (8600), crew (8610), calc engine (8620), TPA simulator (8500)
<code>make run-docpipe, run-vision</code>	Document pipeline (8200) and vision service (8300)
<code>make test / make test-insurer</code>	All hospital-side / insurer-side unit and integration tests
<code>make check-crew</code>	Known-answer check of the insurer crew on the real model
<code>make e2e-hospital / make e2e-full</code>	End-to-end runs: hospital only / hospital to insurer to settlement
<code>make lint, fmt, typecheck</code>	ruff lint, ruff format, mypy
<code>make down / make nuke</code>	Stop the infra containers / also delete all data volumes (asks first)

3.1 Typical session

```
git clone https://github.com/Multi-Agent-MediClaim-Orchestrator/Sohail-Ahmed && cd
Sohail-Ahmed
ollama pull gemma4:latest && ollama pull nomic-embed-text # once
make demo-check # prerequisites
make demo-offline # quick proof (about 1 minute)
make demo # full run on the local model
make demo-down # stop everything
```

3.2 API keys and accounts

No paid API keys are needed

All models run in Ollama on the machine. Every internal secret (database passwords, MinIO keys, HMAC secrets, JWT secrets, n8n encryption key) is generated by `make init-secrets`.

Optional only: a Langfuse key for LLM tracing, and a cloud model key if a faster model is ever wanted; neither is used in the demo.

3.3 Hardware and software requirements

- **Machine:** 16 GB RAM minimum (24 GB+ for the real model), about 20 GB free disk, Apple Silicon or x86-64.
- **Software:** Docker Desktop (host networking on macOS), uv, make, Node 20+, Poppler, Tesseract, Ollama.
- **Ports:** 5432, 5453, 6379, 6333, 8080, 9000/9001, 5688/5689, 8010, 8100-8620, 3100, 3600, 11434 must be free (`make demo-check` verifies).